Bilan de campagne de test

I. Recommandations

A. Tests à automatiser

1. **Le login** : Cette fonctionnalité est un prérequis pour accéder à toutes les autres. C'est un test à exécuter systématiquement, avant tous les autres. Si cette fonctionnalité ne s'exécute pas, toute l'application est inutilisable.
2. **Le panier** : Cette fonctionnalité est complexe, car elle implique le frontend, l'API (Symfony) et la base de données qui doivent rester synchronisées. Un test manuel répété serait fastidieux et sujet à l'erreur humaine. J'ai également constaté qu'un client peut commander un produit qui n'est plus en stock. Si ce bug reste inaperçu dans les prochaines mises à jour, l'impact sur l'activité est direct (produits vendus mais pas en stock, clients mécontents, image de marque dégradée).
3. **Les avis (injection XSS)** : Le champ commentaire de la page Avis est le seul champ de texte libre, donc un point d'entrée classique pour l'injection de contenu malveillant. Une faille de ce type pourrait permettre d'exécuter du code dans le navigateur. Ce test doit vérifier que le contenu soumis est bien neutralisé avant l'affichage.

B. Préconisations pour la suite

- Je recommande d'envisager l'automatisation des tests de l'affichage des produits. Ce scénario n'avait pas été retenu dans cette campagne car il est simple et rapide à vérifier manuellement. Une anomalie d'affichage se constate à l'œil nu, contrairement à la synchronisation du panier ou au login
- Toutefois, cette automatisation deviendrait pertinente si le catalogue des produits venait à évoluer (pagination, filtrage par catégorie, ajout ou mise à jour) afin de garantir la cohérence entre les données de l'API et l'affichage à chaque mise en production.

II. Tests automatisés

Document revu par :

Nom	Fonction	Version	Signature
Fabio	CTO	1.0.1	
Marie	Testeuse manuelle	1.0.1	
Aurélien	Testeur automaticien	1.0.1	

Contexte

Cette campagne de test a été réalisée dans le cadre du projet Eco-Bliss-bath, une application web de vente en ligne de savons et cosmétiques éco-responsables. Trois personnes interviennent sur ce projet : Marie (testeuse manuelle), Fabio (CTO) et moi-même en tant que testeur automaticien.
Cette campagne d'automatisation a représenté environ 15 heures de travail (hors rédaction du présent bilan). Les premiers tests exploratoires manuels ont été réalisés sur Firefox, mais les tests automatisés ont été développés et exécutés avec Cypress sur Chrome.

Objectif(s)

L'objectif principal est d'automatiser les tests pour les fonctionnalités identifiées critiques par l'équipe (Fabio, Marie et moi), ainsi que les 6 requêtes API. Tout cela dans le but de fiabiliser le contrôle qualité du site Eco-Bliss-Bath. Le périmètre de test a été étendu en cours de campagne à un test d'injection XSS sur la page Avis.
Tout en gardant l'objectif mineur d'identifier des anomalies qui n'auraient pas été découvertes par la première campagne de test.

Tests effectués

Tests API (cf cypress/e2e/api/)

Route /login (cf login.cy.ts)

- **POST** avec un utilisateur inconnu : on attend et on obtient un statut 401(Unauthorized).
- **POST** avec un utilisateur reconnu : on attend et on obtient un statut 200, ainsi qu'un token.

Route /orders (cf orders.cy.ts)

- **GET** sans authentification : on attend et on obtient un statut 401 (Unauthorized).
- **GET** avec authentification : on attend et on obtient un statut 200, avec le panier de l'utilisateur connecté.

Route /orders/add (cf orders.cy.ts)

- **PUT** pour un produit disponible : on attend et on obtient un statut 200.
- **PUT** pour un produit en rupture de stock : on attend un statut 400, mais on obtient un statut 200.

Route /products/{id} (cf products.cy.ts)

- **GET :** on attend et on obtient un statut 200, avec les détails du produit correspondant à l'id demandé.

Route /reviews (cf reviews.cy.ts)

- **POST** sans authentification : on attend et on obtient un statut 401.
- **POST** avec authentification : on attend et on obtient un statut 200.

Smoke tests (cf cypress/e2e/smoke/smoke.cy.ts)

- Vérification de la présence des champs email, mot de passe et du bouton de connexion sur la page **/#/login.**
- Vérification de la présence des boutons d'ajout au panier lorsque l'utilisateur est connecté**.**

Tests fonctionnels (cf cypress/e2e/functional/)

- **Connexion (cf login.cy.ts)**
  - **Connexion réussie via le formulaire :** après saisie des identifiants valides et clic sur le bouton de validation, les liens « Mon panier » et « Déconnexion » doivent être visibles.
- **Panier (cf cart.cy.ts)**
  - **Ajout d'un produit disponible au panier :** après sélection d'un produit en stock et ajout au panier, des informations correspondant à ce produit doivent être présentes dans le panier.
  - **Décrémentation du stock après ajout :** le stock affiché sur la fiche produit après ajout doit avoir diminué de la quantité ajoutée.
- **Avis Injection XSS (cf reviews.cy.ts)**
  - **Neutralisation d'un script malveillant** : après soumission d'un avis contenant une charge XSS (<img src=x onerror='alert(« XSS »)'>) aucune alerte ne doit se déclencher et l'attribut « onerror » ne doit plus être présent dans le DOM.

Résultats de tests

La campagne complète a été exécutée via la commande **npx cypress run**. Sur les 15 tests automatisés, 14 sont passés avec succès. Le seul échec correspond à l'ajout d'un produit en rupture de stock, échec attendu car représentant une anomalie réelle de l'application.

Analyse 401/403 sur /orders sans authentification
*Marie attendait un 403 lors d'une requête **GET** sur **/orders** sans authentification mais un code 401 est obtenu.*
D'après mon analyse, je ne retiens pas ce comportement comme une anomalie car aucune donnée d'authentification n'étant fournie, le serveur ne pourra techniquement pas évaluer les droits de l'utilisateur. Un code 401 (échec d'authentification) est donc plus cohérent qu'un 403 (échec d'autorisation qui suppose que l'identité de l'utilisateur ait pu être établie avant la requête et que l'on possède le token).

Rapport d'incident : méthode HTTP incorrecte sur l'ajout au panier

- **Étapes de reproduction :** envoyer une requête **PUT** sur **/orders/add** pour ajouter un produit au panier.
- **Résultat attendu : POST**, conformément aux bonnes pratiques, qui réservent **PUT** au remplacement complet d'une ressource existante et **POST** à la création d'une nouvelle ressource.
- **Résultat obtenu :** la requête utilise la méthode **PUT**.
- **Niveau concerné :** API Symfony.
- **Impact :** l'ajout au panier reste opérationnel donc aucun impact.
- **Criticité :** faible (cette anomalie n'est pas bloquante, mais un correctif est recommandé pour la cohérence et la maintenabilité de l'application).
- **Reproductibilité :** systématique.

Rapport d'incident : absence de contrôle du stock disponible lors de l'ajout au panier

- **Étapes de reproduction :**
  - Se connecter avec un compte valide**.**
  - Envoyer une requête **PUT /orders/add** avec l'id d'un produit dont le stock n'est pas disponible.
- **Résultat attendu :** le serveur doit refuser la requête (code 400).
- **Résultat obtenu :** le serveur accepte la requête et renvoie un code 200. Donc le produit est ajouté au panier malgré l'absence de stock.
- **Niveau concerné :** Front-end et back-end : le front-end n'effectue aucune vérification avant l'envoi de la requête, mais l'absence de validation côté back-end est le point le plus critique.

- **Impact :** un client peut commander un produit indisponible ce qui entraîne une commande impossible à honorer. Ce qui créera une insatisfaction du client et une mauvaise image pour l'entreprise**.**
- **Criticité :** impact direct sur l'activité commerciale, criticité élevée.
- **Reproductibilité :** systématique
- **Capture d'écran** : cf **cypress/screenshots/api/orders.cy.ts**

Test d'injection XSS sur la page Avis : aucune anomalie détectée
Deux injections malveillantes ont été testées sur le champ commentaire (manuellement puis automatiquement) : une balise **<script>** et une **<img>** avec un attribut **onerror**. Dans les deux cas, aucun code n'est exécuté. L'inspection du DOM confirme que la balise **<script>** est entièrement supprimée et l'attribut **onerror** est retiré avant l'affichage. Ce point ne fait donc l'objet d'aucun rapport d'incident.

Smoke tests : aucune anomalie détectée. Les éléments essentiels du parcours utilisateur (champs, bouton de connexion, bouton d'ajout au panier) sont bien présents et affichés.

Confiance

Je n'apporte pas de confiance à cette version pour une mise en production.

L'anomalie critique concernant l'absence de contrôle du stock est bloquante et doit être corrigée en priorité absolue. Un client peut actuellement commander un produit en rupture de stock, sans qu'aucune vérification ne l'en empêche, ni côté interface, ni côté serveur. Les conséquences sont directes et impactantes pour l'image de l'entreprise (commandes impossibles à honorer, remboursements à traiter, mécontentement des clients). Le stock affiché pouvant descendre en valeur négative (jusqu'à -8 constaté sur certains produits lors des tests), l'intégrité des données de stock est compromise, ce qui fausse toute décision (vente, réapprovisionnement, ...) s'appuyant sur ces chiffres. Cette correction doit intervenir côté serveur, afin de garantir un niveau de vérité des stocks, et côté interface pour désactiver la mise dans le panier de produit en rupture de stock.

Deuxièmement, même si l'anomalie n'est pas bloquante, **la correction de la méthode HTTP pour l'ajout au panier** me parait également être à corriger, afin de respecter les bonnes pratiques et de faciliter la maintenabilité du projet pour les futurs développeurs qui pourraient interagir avec l'API à l'avenir. Cette anomalie n'ayant aucun impact fonctionnel constaté, elle peut être traitée lors d'un prochain sprint, une fois l'anomalie de stock résolue.